

A PROJECT REPORT ON

Multiplayer Obstacle Course Game

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

ADITYA MITTAL
PRANAV JAJU
SUNAY BHOYAR
TARUN SANTANI

Exam No: B400050313
Exam No: B400050553
Exam No: B400050344
Exam No: B400050579



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

Multiplayer Obstacle Course Game

Submitted by

ADITYA MITTAL
PRANAV JAJU
SUNAY BHOYAR
TARUN SANTANI

Exam No: B400050313
Exam No: B400050553
Exam No: B400050344
Exam No: B400050579

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. Mayur Chavan** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. Mayur Chavan
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: PICT, Pune
Date: 12/05/2025

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Multplayer Obstacle Course Game**’.*

*We would like to thank our project guide **Prof. Mayur Chavan** as well as subject experts **Dr. Girish Potdar** and **Prof. Madhuri Mane** for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.*

*We are also grateful to **Dr G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Aditya Mittal
Pranav Jaju
Sunay Bhoyar
Tarun Santani
(B.E. Computer Engg.)

ABSTRACT

This project presents the development of a Multiplayer Obstacle Course Game, aimed at delivering an engaging and competitive player experience through real-time multiplayer interaction. The game was developed using Unity, incorporating core components such as networked player synchronization, character control, physics-based obstacles, and real-time communication. In such obstacle games, players navigate chaotic and unpredictable environments while adapting their strategies to avoid elimination and reach the finish line. The development process involved designing level mechanics, implementing collision detection, optimizing game performance, and ensuring reliable multiplayer functionality through Photon Fusion. This project demonstrates a practical application of the principles of multiplayer game development, focusing on technical implementation, user-centric design, and iterative testing to build a fun and scalable multiplayer game environment. The project aimed to blend technical depth with engaging game play, fostering both fun and competitive spirit.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.4	Scope and Limitations	4
1.5	Methodology of Problem solving	6
2	Literature Survey	8
2.1	Literature survey	9
3	Software Requirements Specification	12
3.1	Assumptions and Dependencies	13
3.2	Functional Requirements	14
3.3	External Interface Requirements	16
3.3.1	Hardware Interfaces	16
3.3.2	Software Interfaces	16
3.4	Nonfunctional Requirements	17
3.4.1	Performance Requirements	17
3.5	System Requirements	17
3.5.1	Software Requirements	18
3.5.2	Hardware Requirements	18
4	System Design	19
4.1	Introduction to System Design	20
4.2	Game Architecture	20
4.2.1	Asset Layer (Design& Assets)	20
4.2.2	Game Engine Layer	21
4.2.3	Scripting Layer	22
4.2.4	Networking Layer	22
4.2.5	AI & Pathfinding Module	23
4.2.6	Cross-Platform Considerations	24

4.3	UML Diagrams	25
4.3.1	Use Case Diagram	25
4.4	Class Diagram	26
5	Project Plan	27
5.1	Project Estimate	28
5.1.1	Reconciled Estimates	28
5.1.2	Project Resources	28
5.2	Risk Management	30
5.2.1	Risk Identification	30
5.2.2	Risk Analysis	30
5.2.3	Overview of Risk Mitigation, Monitoring, Management	30
5.3	Project Schedule	32
5.3.1	Project Task Set	32
5.4	Team Organization	33
5.4.1	Team Structure	33
5.4.2	Management Reporting and Communication	33
6	Project Implementation	34
6.1	Overview of Project Modules	35
6.2	Tools and Technologies Used	35
6.3	Model Selection	35
6.4	Algorithm Details	36
6.4.1	AI Target Selection Algorithm (From AIHandler.cs)	36
6.4.2	Room Code Generation (From UIManager.cs)	36
6.4.3	Customization Sync via RPC (From NetworkCustomizationManager.cs)	36
7	Software Testing	37
7.1	Type of Testing	38
7.2	Testing Methodology	38
7.3	Test Cases & Test Results	38
8	Results	41
8.1	Outcomes	42
8.2	Screen Shots	43
9	Conclusions	54
9.1	Conclusions	55
9.2	Future Work	55
9.3	Applications	56

10 Appendix	57
10.1 Appendix A	58
10.1.1 Technical Feasibility	58
10.1.2 Operational Feasibility	58
10.1.3 Economic Feasibility	58
10.2 Appendix B	60
10.2.1 Survey Paper Details	60
10.2.2 Implementation Paper Details	61
10.3 Appendix C	62
10.3.1 Plagiarism Report	62
10.3.2 Event Participation Details	63
11 References	64

List of Figures

4.1 Use case diagram	25
4.2 Class Diagram	26
5.1 Summarized Timeline Chart	33
8.1 The Game title	43
8.2 The main level developed	43
8.3 Main objective of grabbing the boxes	44
8.4 The divine cats where boxes are delivered	45
8.5 Box being delivered	46
8.6 Combat between the players	47
8.7 Knockdown by player	48
8.8 Obstacles in the course	49
8.9 AI Bot using A* for path-finding	50
8.10 The Game Main screen	50
8.11 Player name customization	51
8.12 Player customization	51
8.13 Room Join code screen	52
8.14 Waiting Lobby	52
8.15 Game play while collecting box	53
8.16 Game play in combat with player	53
8.17 In game pause menu	53
10.1 Plagiarism report	62
10.2 INC winners certificate	63

List of Tables

7.1 Functional Test Cases for Unity Multiplayer Game	 39
--	--------------

CHAPTER 1

INTRODUCTION

1.1 Overview

The main goal of this project is to design an enjoyable lighthearted multiplayer obstacle course game where players navigate their way through levels including chaotic moving traps and challenges. Each player character is executed with a funny and unpredictable gameplay due to the purposely awkward and clumsy animations. An additional unique aspect of the game is the blend of competition, strategy and comedy to engage users across the skill spectrum. This report presents the complete iterative development process from early paper sketches and mockups to a fully functional and scalable digital game, focusing on the development workflow. Document critical steps of the uniquely creative process, discerning technical choices and features crafted to entertain users while being marketable in the contemporary gaming landscape.

1.2 Motivation

The global gaming industry is experiencing an incredible boom, with estimates suggesting it could surpass a whopping \$397 billion by 2029. Thanks to advancements in technology, the rise of smartphones, and the growing love for online multiplayer games, gaming has evolved from a niche pastime into a leading form of entertainment worldwide. Among this exciting growth, India shines as the second-largest gaming market in terms of user base, with over 442 million active players—a figure that keeps climbing, especially among Gen Z and millennials.

However, despite this massive player base, India only accounts for about 1.2% of global gaming revenues, which highlights a significant disparity. Indian gamers are some of the most engaged in the world, yet the market heavily relies on games developed abroad. Titles like Call of Duty: Mobile, PUBG, and Clash of Clans are at the top of the charts, but these successes are rarely homegrown. This gap not only represents a missed economic opportunity but also showcases the untapped creative and technical potential that India has to offer.

One major reason for this underperformance is the shortage of skilled game developers, studios, and production pipelines capable of creating content that meets international standards. The Indian gaming ecosystem is still in its early stages, lacking both industry-wide mentorship and strong local infrastructure. Additionally, very few Indian games have managed to make a mark on the global stage—often due to limited marketing, a lack of cultural resonance, or simply not achieving the technical finesse expected in

major markets.

Our motivation for this project is deeply rooted in a desire to change that narrative.

We're in the process of creating a multiplayer obstacle course game from scratch—a wild, humor-filled race packed with unpredictable traps and intentionally clumsy character physics. This project isn't just a fun technical challenge; it's a bold statement of creative independence and innovation. We want to demonstrate that with the right approach, we can carve out a unique space in the gaming world.

1.3 Problem Definition and Objectives

In recent years, multiplayer games have seen a great rise in popularity, mostly by their dynamic gameplay, competitive edge, and a strong sense of community they bring among players. However, developing such games is not a small task where we have to deliver real-time responsiveness, engaging mechanics, and high replay value across a range of devices. The main challenge we aim to address in this project is how to build a visually great, technically robust multiplayer obstacle course game, optimized for diverse hardware conditions, especially even on low-end mobile devices.

Most games in this type of genre are developed by large game developers with no limitations of resources, but through this project, we aim to demonstrate that a even small, focused team can make a high-quality user experience without the backing of a major game development company. Inspired by globally popular games like Fall Guys, our game has a the chaotic fun with obstacles and intentionally clumsy character movement. We plan to take this concept further by introducing betterments in multiplayer mechanics, smoother controls, and better matchmaking, all while ensuring broad device compatibility.

Our primary objective is to design and develop a multiplayer obstacle course game that is both challenging and entertaining, offering players a unique and new experience each time they play. Key features include character customization, dynamically generated obstacles, and real-time multiplayer functionality powered by Unity. Special attention is being given to optimizing performance across devices, ensuring smooth transmission gameplay even on lower-end smartphones, without sacrificing visual responsiveness.

Beyond the technical things, our project emphasizes player engagement, creativity. We aim to build an environment where players can play the way they want, make quick decision and express themselves through gameplay. Additionally, we are documenting our development process and design

choices to serve as a learning resource for other aspiring game developers, especially those working within constraints.

Ultimately, this project is more than just a game but it is a statement of what's possible with passion, focus, and innovation. It showcases not only our technical capabilities, but also our commitment to good game design, player psychology, and contributing to the evolving landscape of Indian game development.

1.4 Scope and Limitations

This project centers on developing a **multiplayer obstacle course game** that delivers chaotic fun, strategic competition, and replayable excitement through dynamic gameplay and real-time player interaction. The game features **clumsy, physics-based character movement**, randomized obstacle layouts, and lighthearted visuals that create an engaging and unpredictable environment. Our scope spans the **complete game development pipeline**, from initial design ideation to deployment, using **Unity** as the primary game engine and **Unity Cloud services** for real-time multiplayer networking and backend infrastructure.

Project Scope

The core components within the project scope include:

- **Game Mechanics Development:** Designing character movement, jumping, sliding, and collision systems with ragdoll-like physics to enhance entertainment value.
- **Level Design:** Building multiple, visually distinct levels populated with traps, moving platforms, rotating hammers, and timed obstacles using Unity's physics system.
- **Multiplayer Implementation:** Integrating matchmaking, player synchronization, and room management using Unity's Netcode for GameObjects or Photon Fusion, with a focus on low-latency communication.
- **Cross-Platform Compatibility:** Developing for both **PC and mobile platforms** (Android/iOS) with responsive controls—supporting touch, keyboard, and potentially gamepad input.

- **UI/UX Design:** Crafting intuitive user interfaces including menus, lobby screens, matchmaking indicators, leaderboards, and post-game stats.
- **Character Customization:** Allowing players to personalize avatars through unlockable skins, costumes, and accessories, enhancing player engagement.
- **Scoring and Reward System:** Designing progression mechanics such as XP, in-game currency, achievement badges, and level-based rewards to promote continuous play.
- **Performance Optimization:** Implementing occlusion culling, texture compression, object pooling, and adaptive quality settings to ensure smooth gameplay on both high- and low-end devices.
- **Testing & Feedback:** Conducting alpha and beta testing cycles to gather user feedback and refine gameplay, controls, and overall balance.
- **Documentation & Learning Material:** Maintaining comprehensive development documentation, design decisions, version tracking, and tool usage to support transparency and learning for future developers.

Limitations

Despite our ambitious goals, the project is constrained by several key limitations:

- **Cross-Platform Challenges:** Ensuring seamless performance and consistent experience across varying hardware specifications is technically complex. Mobile devices, in particular, present challenges with memory management, screen size differences, and input systems.
- **Network Synchronization:** Real-time multiplayer games require highly accurate state synchronization, lag compensation, and data interpolation techniques to avoid desync and jitter—especially in obstacle-heavy environments.
- **Resource Constraints:** As an independent team without studio backing, we face limitations in budget, affecting access to professional-grade assets, premium Unity plugins, and cloud infrastructure.

- **Time Constraints:** Our development timeline is limited, requiring disciplined prioritization, agile sprint cycles, and scope control to meet project goals.
- **Tooling and Infrastructure:** Advanced features such as anti-cheat systems, server-side validation, and cloud save are currently out of scope due to limited infrastructure but are identified as future enhancements.
- **Limited Industry Ecosystem:** Working within India's developing game development ecosystem means reduced access to mentorship, professional guidance, and timely community feedback.
- **Team Bandwidth:** With a small team handling game design, art, networking, and testing, bottlenecks occur during integration and polish phases.

Despite these challenges, our goal is to deliver a **technically sound, creatively rich, and highly playable game** that showcases what small, passionate teams can accomplish. Through modular architecture, reusable components, and user-centric testing, we aim to contribute meaningfully to the Indian and global gaming landscapes.

1.5 Methodology of Problem solving

The development of the Multiplayer Obstacle Course Game followed a structured and iterative problem-solving approach. The process began with a clear understanding of the problem statement which aims to focus on delivering an engaging, competitive multiplayer experience with customizable characters and dynamic obstacles. Next, we conducted a market study and research on game development practices, analyzing existing obstacle-based games to identify common design patterns, gameplay mechanics, and player expectations. This helped us align our goals with industry standards while identifying areas for innovation. We started our basic prototyping by translating paper sketches into 3D assets using Blender, laying the foundation for game scenes and characters. This visual planning phase was crucial for exploring design feasibility and spatial arrangement in obstacle layouts. Once the core visuals were in place, we moved on to defining player mechanics and animations, focusing on smooth character movement, jumping, and interaction with the environment. Realistic and responsive controls were prioritized to ensure an immersive experience. We then implemented character customization features, allowing players to modify their avatars and express individuality in

multiplayer sessions. Following this, we integrated multiplayer room management using Photon Fusion, enabling users to create and join game lobbies seamlessly. To maintain a consistent multiplayer experience, we addressed challenges related to network synchronization, ensuring that character positions, movements, and customizations were accurately reflected across all clients. Finally, we conducted testing among fellow students, collecting feedback on gameplay, UI design, and responsiveness. Based on this feedback, we iteratively improved the user interface and overall user experience, focusing on clarity, intuitiveness, and visual appeal. This problem-solving methodology ensured steady progress, with each development phase building logically on the last and incorporating continuous feedback to refine the final product.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature survey

A multitude of research endeavors have focused on leveraging sophisticated models and innovative approaches to address the complex challenges associated with developing responsive and immersive game experiences in dynamic and real-time environments. This literature survey aims to provide a concise overview of several prominent studies that have explored various methodologies, ranging from deep learning-based frameworks to novel networking and rendering techniques, all aimed at improving the performance, scalability, and adaptability of modern game development systems.

TastyCherryGames. Photon Fusion in Unity: (2024) [1] Provided a study in photon fusion which is a high-performance networking solution developed as part of the Photon Engine suite, specifically tailored for fast-paced, real-time multiplayer games. It distinguishes itself from other networking frameworks like Photon Realtime and Photon Quantum by employing a tick-based deterministic synchronization model and advanced lag compensation mechanisms.

Key features include rollback support for accurate gameplay under latency, predictive input handling, and cloud integration, enabling scalability to thousands of concurrent users. Fusion supports multiple game modes, such as Host, Shared, and Single, with cross-platform compatibility including PC, mobile, and consoles. In Unity, Photon Fusion is integrated via the Network Runner component, which manages network states and communication. Developers implement core multiplayer features using NetworkObject and Networked variables to synchronize state and input. Photon Fusion also provides matchmaking, custom network events, and performance optimizations such as serialization and load balancing. Tools for debugging, logging, and network simulation further assist in optimizing performance under real-world network conditions.

Overall, Photon Fusion offers a comprehensive and developer-friendly environment for building responsive and scalable multiplayer games. Its architecture makes it particularly suitable for competitive gaming where accuracy, speed, and synchronization are paramount.

Booth, J. & Ivanov, V (2020) [2] In recent years, there has been a growing interest in the application of modern optimal control techniques to character animation, driven by advances in policy optimization algorithms,

increased computational capabilities, and improvements in game engines. Several studies have demonstrated that naturalistic character motion can be achieved by combining motion capture data adherence with user input tracking. In this context, one notable work presented an open-source implementation of physics-based character control within the Unity framework, aimed at bridging the gap between academic research and practical application. The implementation is designed to lower the entry barrier while maintaining a steep learning curve, enabling both researchers and developers to experiment with physics-driven character control in interactive environments.

Linåker, J., Bjarnason, E. & Fagerholm, F.(2024) [4] The game industry operates under intense competition, often constrained by short development timelines and limited budgets. In this context, continuous experimentation (CE) has emerged as a vital strategy for validating design decisions and steering development toward market success. One study investigated CE practices specifically within the domain of pre-release indie game development. The researchers conducted exploratory interviews with ten indie developers from various companies and analyzed the data through iterative coding. The study proposed a CE framework that outlines critical considerations for planning and executing experiments. A key observation was that pre-release experimentation largely depends on qualitative feedback due to limited user data availability. The findings also highlight the constraints indie developers face, including restricted participant access and potential audience biases, which can affect the generalizability of results. The insights offered are not only applicable to indie developers but also provide valuable implications for larger studios and software-intensive industries facing similar challenges in early-stage product validation.

Gillberg, Jonas, Bergdahl, Joakim, Sestini, Alessandro, Eakins, Andrew, & Gisslén, Linus. (2023) [9] Transitioning from research to production is a well-recognized challenge, particularly in the context of large-scale and complex software systems such as AAA games. One study explored the integration of reinforcement learning (RL) into an existing automated game testing framework, which previously relied on scripted bots. The objective was to enhance testing capacity and improve coverage across high-profile titles, including *Battlefield 2042* and *Dead Space* (2023). The research documented the technical process of embedding RL into the production pipeline, highlighting the discrepancies between development and runtime environments that complicate such transitions. Additionally, the paper identified several critical bottlenecks encountered during the integration process, offering insights for practitioners aiming to implement similar solutions. To ac-

celerate the adoption of machine learning—particularly reinforcement learning—in game development, the study proposed future research directions that could address current limitations and improve the feasibility of RL in production-level testing environments.

Haworth, Brandon, Berseth, Glen, Moon, Seonghyeon, Faloutsos, Petros, & Kapadia, Mubbasir. (2022) [14] presented Recent advancements in the availability of high-quality motion capture datasets have enabled the development of kinematics-based controllers capable of producing realistic and varied human motion without the need for task-specific goals. One notable study introduced an algorithm for constructing such controllers for physically simulated characters with high degrees of freedom. The proposed method utilizes Conditional Variational Autoencoders (CVAEs) to learn physics-based controllers that can replicate a wide range of behaviors consistent with the training data. These controllers demonstrate strong generalization by maintaining coherent motion over extended periods without requiring goal conditioning, making them suitable for solving complex downstream tasks. The study also includes demonstrations across multiple motion capture datasets and evaluates the algorithm’s components through ablation studies. The associated code and datasets were made publicly available to support further research and development.

Barsode, Arya, Metha, Soham, Shendkar, Nilesh, Jadhav, Rudra, Patil, Ashwini. [19] research has increasingly focused on the development of multiplayer virtual game environments aimed at reducing student stress and enhancing critical thinking abilities. One such study explores the integration of socially engaging multiplayer games to alleviate the psychological burden experienced by students due to academic responsibilities, extracurricular demands, and external coaching. Drawing from the principle that humans are inherently social beings, the study highlights how collaborative gaming environments can foster leadership, teamwork, and logical reasoning skills, all while serving as an effective outlet for stress relief. The research also addresses the post-pandemic challenges of social isolation, proposing multiplayer games as a medium for students across diverse demographics to build social connections and engage meaningfully in a stress-free, immersive environment.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

The success of the proposed project relies on several key assumptions. Firstly, players will have a stable internet connection for real-time multiplayer functionality. Since the game relies on networked interactions using Photon Fusion, it is assumed that game's end-users have reliable internet as poor connectivity could lead to lag, desynchronization, or dropping-out of sessions, which are outside the scope of the game's core logic.

The game is optimized for a small group of players per session to ensure synchronization accuracy and performance. Secondly, Photon Fusion's Free Plan limitations shall not be exceeded, the game currently uses Photon Fusion's free plan, which supports up to 20 concurrent users. It is assumed that during development and early deployment, this limit will not be exceeded. Scaling beyond this would require upgrading to a paid plan or switching to a different networking solution.

The game is built using Unity, so it is assumed that the target platforms (such as Windows, Android, or WebGL-supported browsers) meet the minimum requirements to run Unity applications smoothly. The initial version of the game may not store long-term player progress or stats. It is assumed that local or session-based storage is sufficient for now. Lastly, it is assumed that future updates to Unity or Photon Fusion will maintain backward compatibility or provide migration guides if changes occur.

Following are some of the dependencies which are used for development of this multiplayer game:

- **Unity Game Engine** – Used as the primary development platform for building and testing the game.
- **Photon Fusion SDK** – Provides real-time multiplayer networking and synchronization support.
- **Blender** – Used to design and animate 3D characters and game environment models.
- **Version Control System (e.g., Git + GitHub)** – Enables collaborative development, code tracking, and version management.
- **Testing Devices or Simulators** – Used to test the game across different platforms and ensure cross-device compatibility.

3.2 Functional Requirements

The system shall fulfill the following functional requirements:

1. User Authentication:

- The system shall allow users to register and log in using email or third-party accounts (e.g., Google).
- The system shall validate user credentials securely.

2. Multiplayer Lobby Creation and Matchmaking:

- The system shall allow users to create or join multiplayer lobbies.
- The system shall support matchmaking based on player region and skill levels.

3. Player Controls and Input:

- The system shall enable player movement and actions using virtual joystick and swipe gestures on mobile.
- The system shall support keyboard/gamepad controls on desktop.

4. Obstacle Generation and Gameplay Mechanics:

- The system shall dynamically spawn obstacles and power-ups during gameplay.
- The system shall track player performance and adjust difficulty accordingly.

5. In-Game Communication and Feedback:

- The system shall provide in-game notifications for player progress and eliminations.
- The system shall support basic emoji-based communication.

6. Leaderboard and Scoring:

- The system shall maintain a leaderboard based on players' game performance.
- Scores shall be updated in real-time after each match.

7. Customization and Avatars:

Multiplayer Obstacle Course Game

- The system shall allow users to select and customize avatars before a game session.
- Unlocked items shall persist between sessions.

3.3 External Interface Requirements

3.3.1 Hardware Interfaces

- **Mobile Devices:**
 - Android 10+ devices with minimum 4 GB RAM, OpenGL ES 3.0+ support.
 - iOS devices running iOS 13+.
 - Touchscreen and accelerometer support for gameplay.
- **Desktop Machines:**
 - Windows 10/11 or macOS with 8+ GB RAM, mid-range GPU (e.g., Nvidia GTX 1060 or better).
 - Keyboard/mouse or game controller for input.
- **Game Server:**
 - Dedicated or cloud-hosted server with low-latency network interface (e.g., AWS EC2, Firebase, or Photon).

3.3.2 Software Interfaces

- Unity Game Engine 2022.3 LTS (cross-platform development)
- Photon Unity Networking (PUN) or Unity Netcode for multiplayer support
- Firebase/PlayFab for authentication, leaderboard, and data storage
- REST APIs or WebSockets for server communication
- Android SDK and Xcode for mobile builds
- Visual Studio 2022 with Unity workload for development and debugging

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- The game shall maintain a frame rate of at least **30 FPS on mobile devices** and **60 FPS on desktops** during active gameplay.
- Multiplayer sessions shall maintain a **latency below 150ms** for smooth synchronization between players.
- The application shall **load within 5 seconds** on mobile devices and desktops.
- Resource usage (CPU/GPU/RAM) shall remain within thresholds to prevent overheating or throttling on mobile devices.
- The server shall be capable of supporting **up to 100 concurrent players** across multiple rooms without failure.

3.5 System Requirements

1. Machine 1:

- Windows 11 (64-bit)
- 16 GB System RAM
- 1 TB SSD Storage
- Nvidia RTX 2060 GPU (6GB VRAM)
- Nvidia Driver Version 537
- Unity Editor 2022.3 LTS
- Visual Studio 2022 with Unity Workload

3.5.1 Software Requirements

The game development stack includes the Unity Editor (preferably 2022.3 LTS for stability), C scripts, and the Photon Fusion or Unity Netcode for multiplayer functionality. For backend development, Docker-based server containers are used to simulate real-time game sessions. Continuous Integration tools like GitHub Actions are used for build automation. Remote access tools (like AnyDesk) support collaborative testing and debugging.

3.5.2 Hardware Requirements

The development environment requires a minimum of 16 GB RAM, a dedicated Nvidia GPU with at least 6 GB VRAM for rendering high-fidelity scenes, and an SSD for fast asset loading and game build operations. For server-side simulation, at least 32 GB RAM and 10+ GB VRAM are preferred for testing multiple clients. A high-speed internet connection is critical for real-time multiplayer testing.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction to System Design

The system design of our game focuses on creating an interactive, scalable, and multiplayer-enabled gaming experience using industry-standard tools and practices. The game is developed using Blender for 3D asset modeling, Unity as the core game engine, C# for gameplay scripting, and Photon Engine for multiplayer networking.

The design is modular, with clearly separated components for player control, AI behavior, game logic, and network communication. The system integrates AI agents powered by the A* algorithm to simulate intelligent opponents and includes real-time interaction across platforms through Photon PUN (Photon Unity Networking).

This section provides a detailed breakdown of

- The overall game architecture
- Interactions between players, AI, and server
- UML diagrams such as use case and class diagrams
- Data flow through the system
- Network synchronization mechanisms
- Planned scalability and extension paths, including metaverse integration and reinforcement learning

4.2 Game Architecture

The architecture of our game is structured into four main layers, each responsible for a specific aspect of game functionality. This structure allowed the task distribution and proper management of the game developments which helped us the main the deadlines of the game development

4.2.1 Asset Layer (Design& Assets)

This layer focuses on the development of the building block of the game which include the assets in the game. Generally a single unit of the game element is developed which is reused in in the different contexts

We use the tools like the **Blender** which is an open-source 3D model development tool and where some of the assets are included from the sketchfab they are made sure to have an open source use license, which makes it totally legal to use them.

This layer the characters and the levels which we think about are brought on the computer screens from the pen paper designs. The final product from the layer are generated in the format such as FBX or the OBJ format of the 3 models which are usable in the unity application for the further implementation of logic making the engine tasks.

4.2.2 Game Engine Layer

This layer brings all the things together where the main game loop is designed and the physics mechanics are added to the different elements of the game. The **Unity Engine** is used here to add the logic and the implementation the decision making power for the game. This layer also develops the input handling mechanism which make the the elements in the game to react as per the instruction given by the user

The things which are taken care of in this layer are

- This layer focuses on the Scene Hierarchy which is responsible for management of queuing of the different scenes of the game which helps the user to change the levels in the game and navigate in the game.
- The next thing to work is the rigid body mechanics and the colliders logic which adds the physics part of the game giving it real feel where a object would react with the other objects as if it is in the real world
- The next step is to add the animation controller which will trigger the required animation when a particular action is called
- The Lighting and the Rendering is next thing to be taken care of where the logic for the lighting is provided which makes the levels real that we would feel playing. Here the sources of the lights are declared with their properties such as what is the intensity of the lights and what is the direction in and the positional arguments for the light which needs to rendered out in order to actually make the things implements in the game
- The last part the UI management which the user will actually interact while playing the game. Now its important that every position is designed which can handle every possible situation and can help the user to navigate with and ease in the game

4.2.3 Scripting Layer

This layer brings the game to life by defining the logic, behaviors, and interaction mechanisms that govern how the game elements function. The scripting is primarily done in **C#**, which is well-supported by the Unity Engine. The logic in this layer controls what happens when a player moves, interacts with an object, or when an event is triggered in the game. It serves as the brain of the entire system.

The things which are taken care of in this layer are:

- **Player Controls:** This section handles the movement, jump, collect, and interaction actions performed by the player. It maps the player's inputs from keyboard, joystick, or touch to in-game behaviors.
- **AI Behaviors:** The logic for non-player characters (NPCs) and bots is defined here. Our AI bots use the *A-Star Algorithm* to calculate the shortest path to targets while avoiding obstacles, simulating intelligent navigation.
- **Game State Management:** The game manager script controls the transitions between different game states such as Start, Play, Pause, Game Over, and Restart. It also manages timers, scoring systems, and win/lose conditions.
- **Object Interactions:** This portion of the script governs how different in-game objects behave upon interaction, such as collecting a box, updating score, or triggering animations and sound effects.
- **UI Logic:** Behind every visual UI element lies a script that handles its functionality – whether it's starting the game, changing scenes, updating health bars, or showing popups.
- **Event Handling:** Custom events and triggers are scripted to notify various parts of the system when a certain action happens – such as a collision, a level completion, or an item pickup.

4.2.4 Networking Layer

The Networking Layer enables the multiplayer functionality of the game using the **Photon Unity Networking (PUN)** framework. This layer ensures that multiple players can connect, interact, and synchronize in real time within the same game environment, regardless of the platform they use.

Photon abstracts away low-level networking complexities and provides a simple API to create, join, and manage game rooms over the internet, making it ideal for rapid development of online games.

The things which are taken care of in this layer are:

- **Room Management:** Players can either create or join existing rooms using Photon. Each room acts as a separate session where a specific number of players can interact.
- **Player Synchronization:** Photon ensures each player's position, animation state, and actions are synchronized across all connected clients using `PhotonView` components and serialization.
- **Remote Procedure Calls (RPCs):** Certain game actions, like collecting a box or triggering an AI response, need to be executed on all clients. Photon RPCs allow the invocation of functions across the network seamlessly.
- **Master Client Model:** One player is assigned as the Master Client, who controls important game decisions such as starting the game, spawning objects, or ending the session. This ensures centralized authority and avoids conflicts.
- **Latency Management:** Although handled internally, Photon provides tools to manage lag, ping, and lag compensation to maintain smooth gameplay.
- **Cross-Platform Connectivity:** Photon supports real-time interaction between players on Android, Windows, and WebGL, ensuring a true cross-platform multiplayer experience.

4.2.5 AI & Pathfinding Module

The AI & Pathfinding Module governs the behavior and decision-making of non-player characters (NPCs) in the game. In our game, AI-controlled bots act as adversaries or collectors that interact dynamically with the environment and players. The primary algorithm powering these bots is the **A* (A-Star) Algorithm**, which is widely used for optimal pathfinding in games.

This module allows AI agents to intelligently navigate toward goals (such as boxes or players) while avoiding obstacles and responding to changes in the environment.

The things which are taken care of in this layer are:

- **Target Acquisition:** Each bot dynamically selects a target (e.g., the nearest collectible box or a player) using distance calculations or priority rules.
- **Path Calculation:** The A* algorithm computes the shortest and most efficient path from the bot's current position to the target, considering obstacles like walls, hazards, or other bots.
- **Obstacle Avoidance:** Bots actively detect and avoid static and dynamic obstacles by recalculating their path if the current one is blocked or suboptimal.
- **Behavior Trees:** The decision logic behind bot actions—like chasing a player, collecting a box, or patrolling—is controlled using basic behavior tree structures or finite state machines (FSM).
- **Adaptive Difficulty:** The AI system can be enhanced to adjust the bots' speed, intelligence, or aggression based on the player's performance, ensuring balanced gameplay.
- **Performance Optimization:** Pathfinding operations are optimized using Unity's NavMesh system, allowing precomputed walkable surfaces and faster navigation for bots in complex scenes.

4.2.6 Cross-Platform Considerations

Cross-platform compatibility is a critical aspect of our game design, allowing seamless access across devices such as Android smartphones, Windows PCs, and WebGL-supported browsers. Unity's robust build system enables deployment to multiple platforms with minimal platform-specific changes. Input handling is abstracted to account for both touch interfaces (on mobile) and keyboard or mouse inputs (on PC), ensuring a consistent gameplay experience. Additionally, Photon Networking supports real-time multiplayer interactions across devices, making cross-device synchronization smooth and responsive. Performance optimization techniques such as texture compression, reduced polygon models, and platform-dependent quality settings are also implemented to maintain visual fidelity and responsiveness across a wide range of hardware configurations.

4.3 UML Diagrams

4.3.1 Use Case Diagram

The Use Case Diagram in Figure 4.1 illustrates how various actors interact with the system to achieve their goals within the game architecture. The primary actors include Players, AI Bots, and the Game Server. Players can log in, control their characters, collect boxes, deliver them to divine entities, and engage in multiplayer sessions. AI Bots are designed to navigate the environment using the A* algorithm, track players or objectives, and dynamically interact with them. The Game Server, powered by Photon, manages matchmaking, real-time state synchronization, and multiplayer coordination. This diagram helps in understanding the dependencies and interactions among the system components, providing a foundation for further system behavior modeling and design implementation.

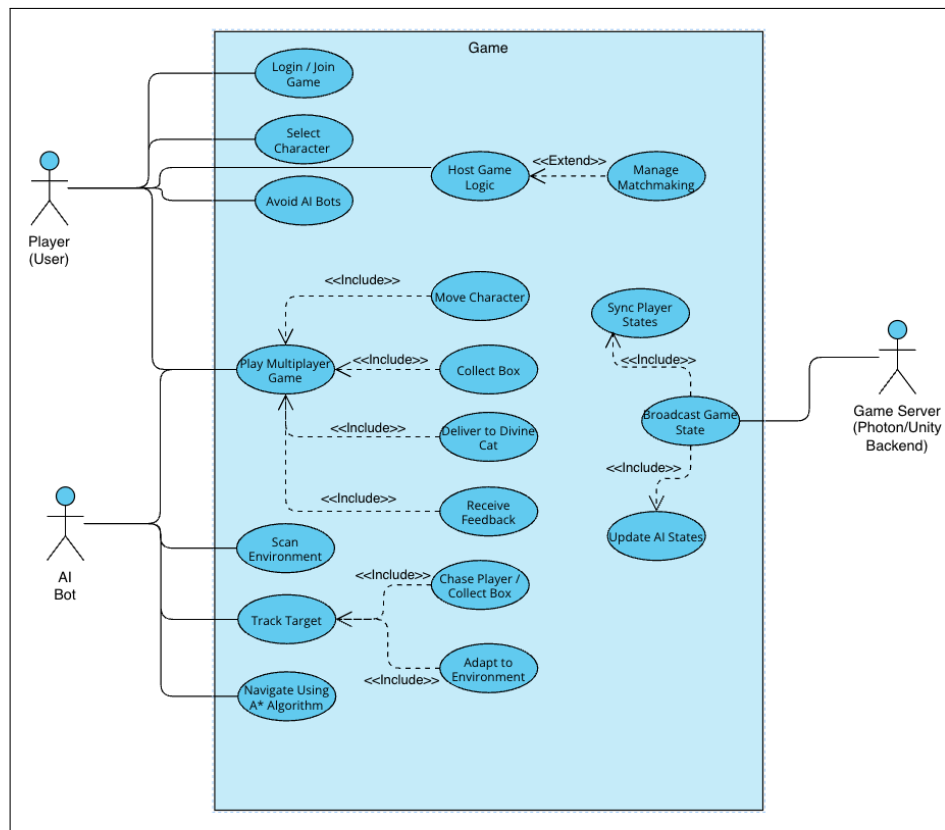


Figure 4.1: Use case diagram

4.4 Class Diagram

Class diagrams are the blueprints of the system/subsystem. Figure 4.2 models the objects that make up the system, displays the relationships between the objects, and describes what those objects do and the services that they provide.

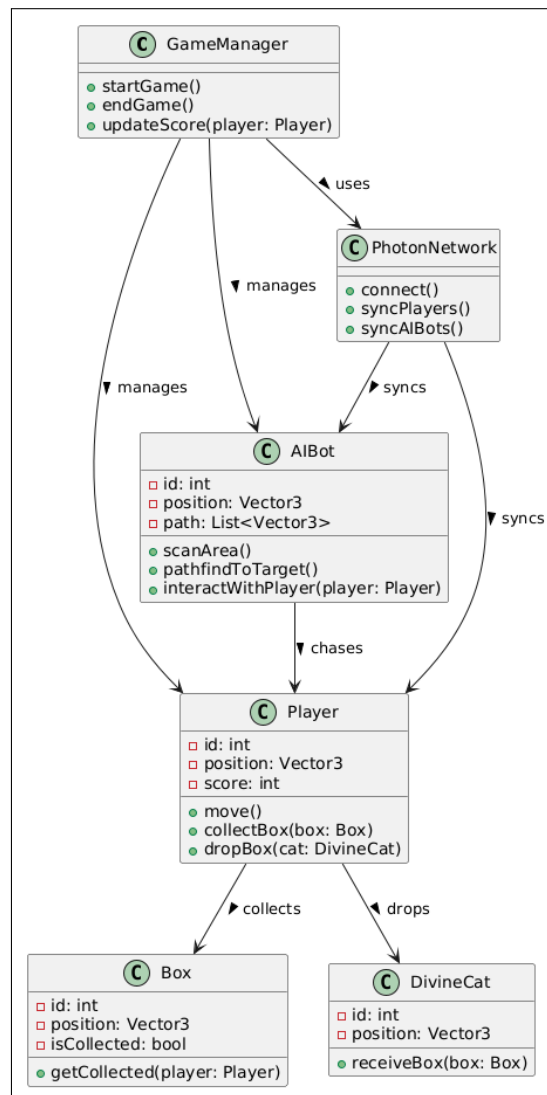


Figure 4.2: Class Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Game Design and Development:** Estimated a total of 3 months to develop the core mechanics, implement active ragdoll physics, and create competitive level designs. This includes integrating online multiplayer functionality and designing systems to ensure smooth gameplay across various network conditions.
- **AI Integration and Testing:** Estimated 2 months to develop and integrate AI players that can substitute for missing players, ensuring a consistent 4-player experience. Extensive testing will be conducted to balance AI behavior and evaluate system performance.
- **Documentation and Review:** Estimated 1 month to provide detailed documentation on architecture, development processes, and AI logic. Regular reviews and updates will be conducted to gather feedback from testers and refine gameplay. Additional documentation will be created for future scalability and possible platform expansions.
- **Quality Assurance and User Feedback:** Focused 1-month effort on playtesting across different levels and network conditions. This includes gathering structured feedback from early users, identifying bugs or balance issues, and iteratively refining gameplay mechanics, AI behavior, and user interface for an engaging and stable multiplayer experience.

Based on these estimates, the total project timeline for developing the multiplayer game is approximately 7 months. Actual timelines may vary depending on technical challenges, team size, and resource availability.

5.1.2 Project Resources

1. **Game Developers (3):** Tasked with the end-to-end development of the game's core features using Unity. Their responsibilities include designing and implementing competitive level logic, player controls, and physics-based interactions using active ragdoll systems to enhance realism and unpredictability. They handle the integration of multiplayer networking using Unity Netcode or Photon, ensuring smooth synchronization of game state across clients. Developers also manage performance optimization for both gameplay and network efficiency,

including load balancing, lag compensation, and platform-specific refinements. Additionally, they collaborate closely with the AI engineer to ensure seamless player handoffs between human and AI participants and maintain a responsive, immersive user experience.

2. **Level and UI/UX Designer (1):** Focuses on crafting engaging level layouts and intuitive player interfaces that support both competitive gameplay and seamless user interaction. Responsibilities include designing obstacle-based levels that complement the active ragdoll physics and encourage strategic movement and interaction. The designer ensures that visual cues, HUD elements, and feedback systems guide players effectively through the game flow. This role also includes prototyping wireframes, conducting usability testing, and iterating designs based on player feedback to enhance clarity, responsiveness, and immersion. Collaboration with game developers ensures that design intentions are preserved during implementation and gameplay remains both functional and aesthetically appealing across all levels.
3. **Project Management Tool:** Trello / Notion
4. **Collaboration Tools:** GitHub, Unity Collaborate, Discord, Google Meet

5.2 Risk Management

5.2.1 Risk Identification

- Technical Risk:
 1. Integration of active ragdoll physics with real-time multiplayer gameplay could lead to instability or performance issues.
 2. Synchronization challenges in maintaining consistent game state across multiple clients.
 3. AI behavior may not scale well in dynamic multiplayer scenarios, leading to unbalanced gameplay.
 4. Network-related issues like latency, packet loss, and server downtime affecting player experience.
- Design Risk: Gameplay mechanics may not feel intuitive or competitive enough, affecting user engagement and retention.
- Operational Risk: Shifts in scope or design direction based on feedback could disrupt timelines or increase workload.

5.2.2 Risk Analysis

- Technical Risk: Assessed as high impact with medium likelihood. Issues with networking and physics integration can severely impact game quality. Mitigation involves modular development, early prototyping, and continuous playtesting.
- Design Risk: Evaluated as medium impact but with high likelihood, especially in early stages. This will be addressed by regular user testing and iterative design updates.
- Operational Risk: Medium impact with medium likelihood. This will be mitigated by defining a flexible development roadmap and maintaining detailed documentation to manage evolving requirements.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- Technical Risk Mitigation: Develop and test ragdoll and multiplayer components independently before integration. Use Unity's profiling

tools and simulation environments to identify bottlenecks. Establish CI/CD pipelines for frequent builds and automated testing.

- Design Risk Mitigation: Conduct frequent internal playtests and external beta testing. Collect actionable feedback through surveys and gameplay analytics to refine controls, UI, and game balance.
- Operational Risk Mitigation: Maintain a clear but adaptable project scope. Use agile methodologies with biweekly sprints and retrospectives to accommodate necessary design or feature changes efficiently.
- Risk Monitoring: Weekly sync meetings will include a dedicated risk check-in. Key risks will be logged and updated in a shared risk register to ensure visibility.
- Risk Management: Development milestones will include built-in buffers. A flexible task allocation plan will help reassign work in case of road-blocks, ensuring consistent project momentum.

5.3 Project Schedule

5.3.1 Project Task Set

- Set up the Unity development environment, version control, and collaboration tools.
- Design core gameplay mechanics, define level structures, and establish system architecture for ragdoll physics and networking.
- Research and evaluate various techniques for implementing active ragdoll physics and real-time multiplayer support using Unity Netcode or Photon.
- Develop and prototype character movement and physics interactions.
- Build the initial networked multiplayer framework to synchronize player actions and game states.
- Design AI player behavior models to fill in for unavailable human players.
- Integrate AI players into multiplayer matches and ensure seamless hand-off between human and AI control.
- Conduct iterative playtests to evaluate game balance, physics behavior, and network performance.
- Optimize performance for both gameplay and networking across different devices and connection speeds.
- Polish UI/UX, animations, and in-game feedback systems to improve player experience.
- Conduct closed alpha testing with a select group of players, collect feedback, and refine game mechanics accordingly.
- Prepare documentation on system architecture, AI logic, and multiplayer implementation.
- Plan for public beta release and final polish based on testing insights.

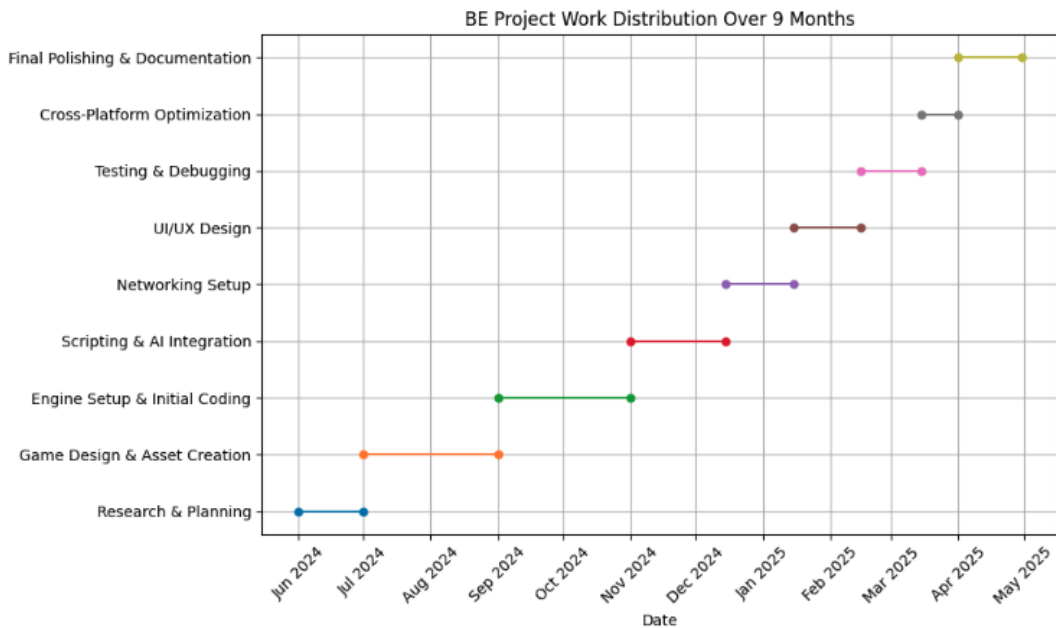


Figure 5.1: Summarized Timeline Chart

5.4 Team Organization

5.4.1 Team Structure

- **Aditya Mittal:** Game Developer – Player Mechanics, AI Design.
- **Sunay Bhoyar:** Game Developer and UI/UX Designer – Level Design, Gameplay UI.
- **Tarun Santani:** Game Developer – Core Multiplayer, Character Customization.
- **Pranav Jaju:** Game Developer – Game Optimization, Core Multiplayer Design.
- **Mr. Mayur Chavan:** Internal mentor and development guide.

5.4.2 Management Reporting and Communication

- **Management:** Git and GitHub handled version control and collaborative development, while Discord and Google Meet facilitated real-time communication and regular stand-up meetings.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- I **Networking and Room Management Module** – Responsible for room creation, joining, and player spawning using Photon Fusion. It uses session codes to manage multiplayer rooms and handles both host and client logic.
- II **UI Management Module** – Manages all user interface panels such as name input, room code generation, and customization menu. Ensures smooth user interaction and stores player preferences using Unity’s PlayerPrefs.
- III **AI Behavior Module** – Implements rule-based AI logic using Unity NavMesh. AI agents autonomously seek out nearby items or ragdolled players and make pathfinding decisions in real time.
- IV **Character Customization Module** – Allows players to customize their avatars, including helmet models and skin colors. These settings are synchronized over the network using Photon Fusion RPCs to ensure all clients see the same configuration.

6.2 Tools and Technologies Used

- Unity 2022.3
- Photon Fusion SDK
- C# (C-Sharp)
- Unity NavMesh
- TextMeshPro
- Unity Test Framework (NUnit)
- Git and GitHub

6.3 Model Selection

Although the project doesn’t use a traditional machine learning model, the AI logic in the game was crafted using rule-based logic for real-time pathfinding and decision-making. This was preferred due to its low latency, deterministic behavior, and integration with Unity NavMesh. AI decisions include

targeting nearby players or items and path planning based on real-time game state.

6.4 Algorithm Details

6.4.1 AI Target Selection Algorithm (From AIHandler.cs)

```
GameObject closestPlayer = potentialPlayers
    .Where(p => p != gameObject)
    .Select(p => new { PlayerObj = p, PlayerComponent =
        p.GetComponent<NetworkPlayer>() })
    .Where(p => p.PlayerComponent != null &&
        p.PlayerComponent.IsActiveRagdoll)
    .OrderBy(p => Vector3.Distance(transform.position,
        p.PlayerObj.transform.position))
    .Select(p => p.PlayerObj)
    .FirstOrDefault(p => Vector3.Distance(transform.position,
        p.transform.position) <= 5);
```

6.4.2 Room Code Generation (From UIManager.cs)

```
private static string GenerateRandomString(int length) {
    const string chars = "0123456789";
    return new string(Enumerable.Repeat(chars, length)
        .Select(s => s[Random.Range(0, s.Length)]).ToArray());
}
```

6.4.3 Customization Sync via RPC (From NetworkCustomizationManager.cs)

```
[Rpc(RpcSources.InputAuthority, RpcTargets.StateAuthority)]
private void RPC_SetCustomization(AppearenceDetails detail, int id) {
    // Synchronizes customization data to all clients
}
```

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

- Unit Testing – Tests individual components like UI button handlers, AI logic, and RPC-based customization. Ensures scripts such as `UIManager.cs` and `AIHandler.cs` behave as expected in isolation.
- Integration Testing – Validates the interaction between different modules such as UI + Networking, or AI + Game World.
- Functional Testing – Verifies game functionality such as player joining rooms, AI navigation, and UI transitions meet the requirements.
- UI Testing – Confirms correct activation of panels and buttons based on user inputs.
- Multiplayer Testing – Ensures synchronization across clients, including character appearance and room joining logic.

7.2 Testing Methodology

We adopted a hybrid testing methodology using automated and manual strategies. Automated unit tests were written using Unity's Test Framework (NUnit) to verify UI logic and AI behavior. Manual testing was performed using Unity Play Mode across multiple clients for multiplayer scenarios.

Each feature was validated against expected outputs. Persistent settings (e.g., player name, customization) were tested using `PlayerPrefs`. Networking RPCs were validated by joining two or more clients and observing synchronized outcomes. We used logs, on-screen UI confirmations, and RPC callbacks to verify logic.

7.3 Test Cases & Test Results

Sample Unit Test Code

`UIManagerTests.cs`

```
[Test]
public void PlayerNameIsSavedAndDisplayed() {
    var inputField = new TMP_InputField();
    inputField.text = "Shraddha";
    PlayerPrefs.SetString("PlayerName", inputField.text);
}
```

Table 7.1: Functional Test Cases for Unity Multiplayer Game

Feature	Input/Scenario	Expected Result	Status
Room Join	Valid Room Code + Player Name	Player joins room successfully	Pass
Room Join	Empty Room Code or Name	Error message shown, join disabled	Pass
Player Name Save	Enter name 'Shrad-dha'	Saved in PlayerPrefs and displayed	Pass
Customization Sync	Set Helmet ID = 1	All clients reflect new helmet	Pass
AI Targeting	Ragdoll within 5 units	AI targets the ragdoll	Pass
Path Debug	Set AI target	Debug lines drawn on NavMesh path	Pass
UI Navigation	Click 'Join Room' button	Panel transitions to Join Room UI	Pass
Helmet Change Button	Call HelmetModuleUp()	Helmet index updates visually	Pass

```
PlayerPrefs.Save();
Assert.AreEqual("Shraddha", PlayerPrefs.GetString("PlayerName"));
}
```

AIHandlerTests.cs

```
[Test]
public void AICanCalculateDistanceToTarget() {
    var ai = new GameObject("AI");
    ai.AddComponent<NavMeshAgent>();
    var aiHandler = ai.AddComponent<AIHandler>();
    GameObject target = GameObject.CreatePrimitive(PrimitiveType.Cube);
    target.transform.position = new Vector3(10, 0, 0);
    float dist = Vector3.Distance(ai.transform.position,
    target.transform.position);
    Assert.GreaterOrEqual(dist, 0);
}
```

CHAPTER 8

RESULTS

8.1 Outcomes

The outcomes of the project address several significant challenges that were successfully navigated:

1. **Balancing Fairness in Multiplayer Settings:** The project successfully implemented dynamic obstacle scaling and adaptive difficulty to ensure fairness, providing all players with an equitable and engaging gameplay experience regardless of skill level or connection quality.
2. **Real-time Synchronization Across Devices:** Despite the challenge of maintaining consistent game states across multiple devices, the project achieved smooth real-time synchronization using optimized netcode and interpolation strategies, ensuring seamless multiplayer interactions.
3. **Low Performance on Mid-Range Mobile Devices:** The game was optimized to run on mid-range devices by incorporating level-of-detail (LOD) rendering, object pooling, and memory management techniques, enabling a wide range of users to enjoy the experience without compromising performance.
4. **Designing a Visually Appealing Yet Lightweight Environment:** The team created cartoon-style low-poly environments and characters that minimized GPU strain while maintaining an engaging and aesthetically pleasing look, aligning with the fun and chaotic theme of obstacle-based gameplay.
5. **Player Retention and Engagement:** The game integrated progressive unlocks, random obstacle layouts, and short, high-reward matches to enhance replay value and maintain player engagement over time, a core requirement for success in the battle royale obstacle genre.

8.2 Screen Shots



Figure 8.1: The Game title



Figure 8.2: The main level developed

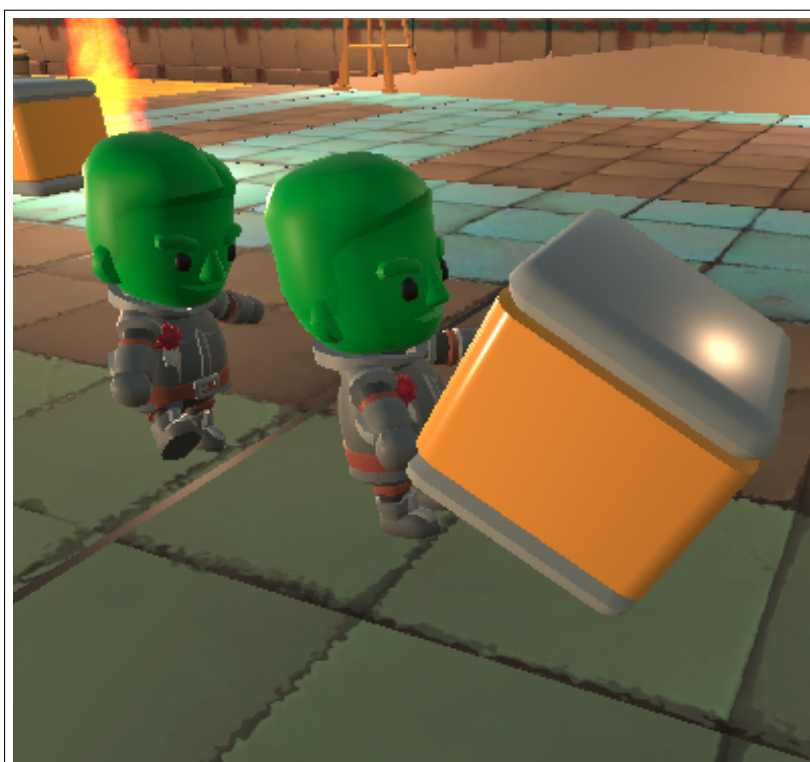


Figure 8.3: Main objective of grabbing the boxes



Figure 8.4: The divine cats where boxes are delivered



Figure 8.5: Box being delivered



Figure 8.6: Combat between the players



Figure 8.7: Knockdown by player



Figure 8.8: Obstacles in the course



Figure 8.9: AI Bot using A* for path-finding

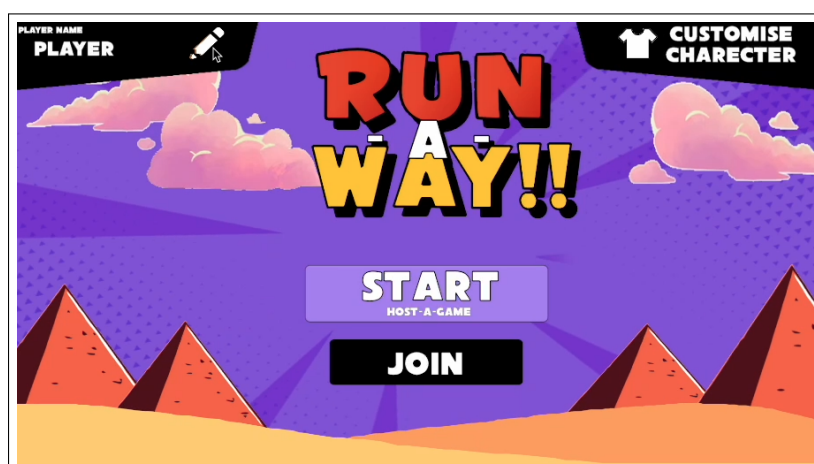


Figure 8.10: The Game Main screen

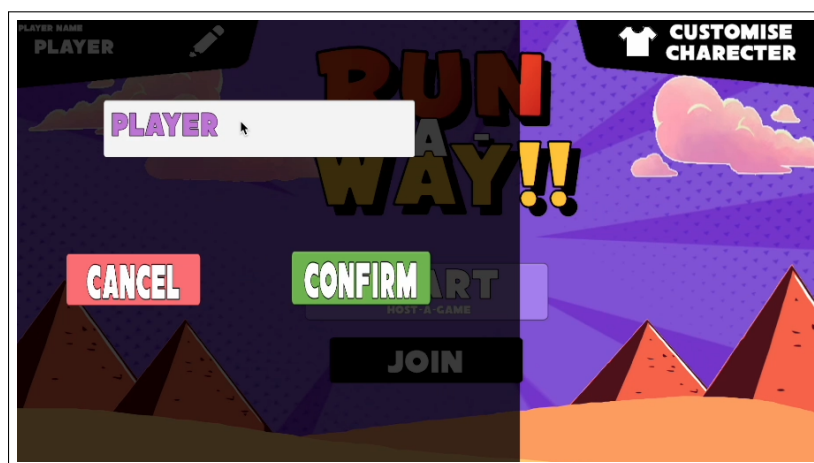


Figure 8.11: Player name customization



Figure 8.12: Player customization

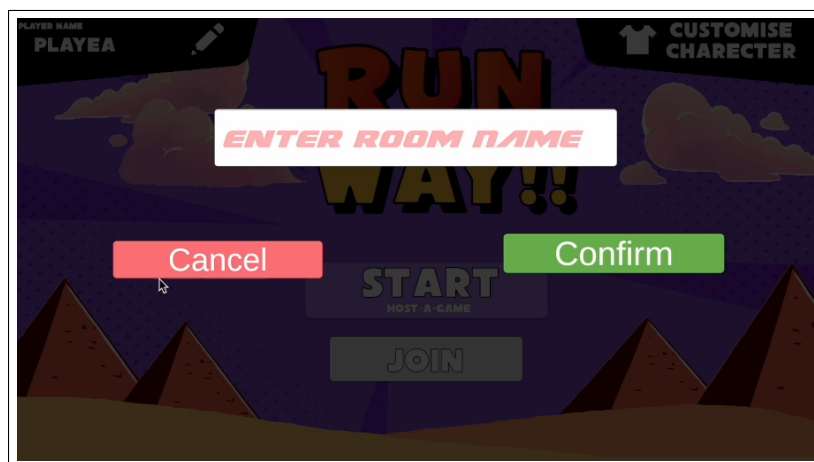


Figure 8.13: Room Join code screen



Figure 8.14: Waiting Lobby



Figure 8.15: Game play while collecting box



Figure 8.16: Game play in combat with player

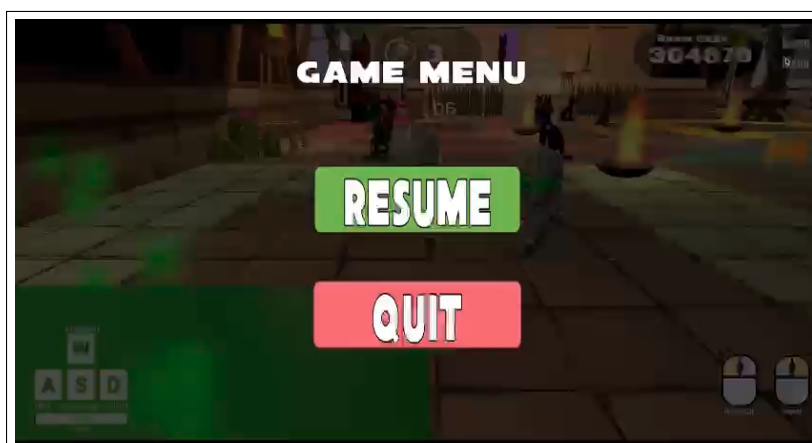


Figure 8.17: In game pause menu

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

In conclusion, the development of the multiplayer obstacle course game successfully demonstrates the feasibility of delivering a fun, fair, and optimized gaming experience on resource-constrained platforms. The game incorporates key innovations such as dynamic obstacle difficulty adjustment, performance optimization for mid-range mobile devices, and effective network synchronization for real-time multiplayer gameplay. These features contribute to an engaging user experience while maintaining system efficiency. Additionally, the game architecture lays a solid foundation for future enhancements such as improved player matching algorithms, scalable level design, and adaptive narrative mechanics. Overall, this project advances the scope of real-time multiplayer games, emphasizing accessibility, fairness, and replayability.

9.2 Future Work

While the current implementation is functional and engaging, several enhancements can be explored to enrich gameplay and scalability:

1. **Cross-Platform Play:** Implementing support for both mobile and PC users in shared lobbies will expand the player base and enable broader community engagement.
2. **AI Bots for Sparse Matches:** Introducing adaptive AI opponents can help maintain match consistency and reduce waiting times during periods of low player availability.
3. **Matchmaking and Ranking System:** Developing an Elo-based or performance-tiered matchmaking system will ensure balanced player matchups and foster competitive fairness.
4. **Advanced Analytics Dashboard:** Building a real-time analytics system to track player behavior, engagement patterns, and drop-off points will guide continuous game improvements.
5. **AR/VR Integration:** Experimental development toward immersive gameplay using augmented or virtual reality can provide an innovative edge for future versions.

9.3 Applications

The developed game system has several applications and broader impacts:

1. **Entertainment and Esports:** The game serves as a platform for casual fun and competitive play, making it suitable for online tournaments and live-streamed esports events.
2. **Game Design Education:** The modular structure and scalable obstacle system can be used as an educational tool in game development and design courses.
3. **Testing AI in Dynamic Environments:** The unpredictable nature of obstacle movement and multiplayer interactions makes the game a suitable testbed for reinforcement learning and real-time AI research.
4. **Community Engagement and Monetization:** The game framework supports in-game rewards, skin unlocks, and seasonal challenges, enabling monetization strategies and fostering a strong community.

CHAPTER 10

APPENDIX

10.1 Appendix A

10.1.1 Technical Feasibility

- The project utilizes Unity 2022.3 LTS, a widely supported game engine, ensuring robust development and deployment across mobile and desktop platforms.
- Integration with Photon Networking or Unity Netcode provides scalable multiplayer capabilities, demonstrating the technical feasibility of supporting real-time online gameplay.
- Cross-platform support with Android, iOS, and Windows is enabled through Unity's build system, simplifying deployment and ensuring wide accessibility.
- Mobile-optimized rendering pipelines and lightweight shaders confirm the technical viability of maintaining performance across devices with varying specifications.

10.1.2 Operational Feasibility

- The game is designed to be intuitive and accessible, with user-friendly controls on mobile (joystick, swipe) and desktop (keyboard/gamepad), ensuring smooth onboarding for players.
- Early-stage testing and feedback sessions confirm that core gameplay mechanics, UI responsiveness, and performance metrics meet user expectations.
- The use of remote logging and crash analytics tools like Firebase ensures effective issue tracking and support, increasing operational efficiency and maintainability.
- The modular architecture of the game allows for easy updates, bug fixes, and feature expansion, supporting ongoing operations.

10.1.3 Economic Feasibility

- The use of free or open-source tools such as Unity (Community License), Firebase, and Photon (free tier) significantly reduces development and infrastructure costs.

Multiplayer Obstacle Course Game

- The game targets monetization through in-app purchases and optional ads, offering potential revenue streams without upfront user charges.
- Cloud-based solutions reduce the need for physical infrastructure, lowering operational costs.
- Given the scalability of the design and potential user engagement, the return on investment from time and resource commitment is economically justified.

10.2 Appendix B

10.2.1 Survey Paper Details

Paper Status: Published

Name of Journal: TIJER, Vol 12 Issue 1, January-2025

Paper Link: <https://tijer.org/tijer/papers/TIJER2501095.pdf>

10.2.2 Implementation Paper Details

Paper Status: Under Review

Name of Conference: NA

Paper Title: Implementation of a Multiplayer Active Ragdoll Game Using Unity Physics Engine

10.3 Appendix C

10.3.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith.

Similarity: 4%

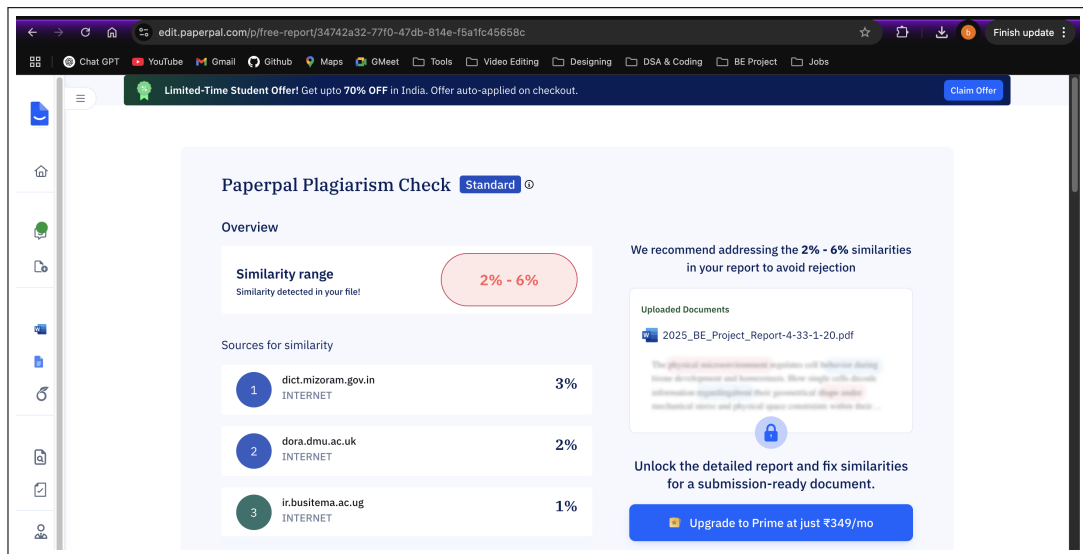


Figure 10.1: Plagiarism report

10.3.2 Event Participation Details



Figure 10.2: INC winners certificate

CHAPTER 11

REFERENCES

- [1] TastyCherryGames. Photon Fusion in Unity: A Comprehensive Guide. TastyCherryGames, 2024. Available: <https://tastycherrygames.com/2024/08/05/photon-fusion-in-unity-a-comprehensive-guide/>
- [2] Booth, J. & Ivanov, V. Realistic Physics Based Character Controller. arXiv preprint arXiv:2006.07508, 2020.
- [3] Unity Technologies. Game Development Testing and QA Best Practices. Unity, 2023. Available: <https://unity.com/how-to/testing-and-quality-assurance-tips-unity-projects>
- [4] Linåker, J., Bjarnason, E. & Fagerholm, F. Pre-Release Experimentation in Indie Game Development: An Interview Survey. arXiv preprint arXiv:2411.17183, 2024.
- [5] Bergdahl, J., Gordillo, C., Tollmar, K. & Gisslén, L. Augmenting Automated Game Testing with Deep Reinforcement Learning. arXiv preprint arXiv:2103.15819, 2021.
- [6] Santos, R., Magalhaes, C., Capretz, L. F., Neto, J. C., Bueno da Silva, F. Q. & Saher, A. Computer Games Are Serious Business and so is their Quality: Particularities of Software Testing in Game Development from the Perspective of Practitioners. arXiv preprint arXiv:1812.05164, 2018.
- [7] Holmgård, Christoffer, Green, Michael Cerny, Liapis, Antonios, & Torgelius, Julian. Automated Playtesting with Procedural Personas through MCTS with Evolved Heuristics. arXiv preprint arXiv:1802.06881, 2018.
- [8] Balyo, Tomáš, Youngblood, G. Michael, Dvořák, Filip, Chrpá, Lukáš, & Barták, Roman. On Automating Video Game Regression Testing by Planning and Learning. arXiv preprint arXiv:2402.12393, 2024.
- [9] Gillberg, Jonas, Bergdahl, Joakim, Sestini, Alessandro, Eakins, Andrew, & Gisslén, Linus. Technical Challenges of Deploying Reinforcement Learning Agents for Game Testing in AAA Games. arXiv preprint arXiv:2307.11105, 2023.
- [10] Zhu, Qingxu, Zhang, He, Lan, Mengting, & Han, Lei. Neural Categorical Priors for Physics-Based Character Control. arXiv preprint arXiv:2308.07200, 2023.
- [11] Xu, Pei, Xie, Kaixiang, Andrews, Sheldon, Kry, Paul G., Neff, Michael, McGuire, Morgan, Karamouzas, Ioannis, & Zordan, Victor. AdaptNet:

- Policy Adaptation for Physics-Based Character Control. arXiv preprint arXiv:2310.00239, 2023.
- [12] Juravsky, Jordan, Guo, Yunrong, Fidler, Sanja, & Peng, Xue Bin. PADL: Language-Directed Physics-Based Character Control. ACM SIGGRAPH Asia, 2022.
- [13] Wang, Tingwu, Guo, Yunrong, Shugrina, Maria, & Fidler, Sanja. UniCon: Universal Neural Controller For Physics-Based Character Motion. arXiv preprint arXiv:2011.15119, 2020.
- [14] Haworth, Brandon, Berseth, Glen, Moon, Seonghyeon, Faloutsos, Petros, & Kapadia, Mubbasir. Physics-Based Character Controllers Using Conditional VAEs. ACM Transactions on Graphics, 2022.
- [15] Holmgård, Christoffer, Green, Michael Cerny, Liapis, Antonios, & Togelius, Julian. Automated Playtesting with Procedural Personas through MCTS with Evolved Heuristics. arXiv preprint arXiv:1802.06881, 2018.
- [16] Tabiszewski, Marek. A Comparative Analysis of Transitions Generated Using the Unity Game Development Platform. Journal of Computer Sciences Institute, vol. 30, pp. 47–52, 2024. DOI: <https://doi.org/10.35784/jcsi.5442>.
- [17] Hussain, Faizan. Unity Game Development Engine: A Technical Survey. University of Sindh Journal of Information and Communication Technology, vol. 4, no. 2, pp. 73–81, 2020. Retrieved from <http://sujo.usindh.edu.pk/index.php/USJICT/article/view/1800>.
- [18] McMahan, Ryan P., Chawla, Nayan N., Cassell, Christian S., Lee, Christopher Peerapon. Virtual Reality Games: Extending Unity Learn Games to VR. arXiv preprint arXiv:2410.16061, 2024. Available at <https://arxiv.org/abs/2410.16061>.
- [19] Barsode, Arya, Metha, Soham, Shendkar, Nilesh, Jadhav, Rudra, Patil, Ashwini. Game Development Using Unity Game Engine for Developing Critical Thinking Skills. International Journal of Advanced Research in Computer and Communication Engineering, vol. 13, no. 2, 2024. DOI: <https://doi.org/10.17148/IJARCCE.2024.13236>.
- [20] Pachoulakis, Ioannis, Pontikakis, Georgios. Combining Features of the Unreal and Unity Game Engines to Hone Development Skills. arXiv preprint arXiv:1511.03640, 2015. Available at <https://arxiv.org/abs/1511.03640>.

LIST OF ABBREVIATIONS

Abbreviation	Illustration
AI	Artificial Intelligence
UI	User Interface
UX	User Experience
3D	Three-Dimensional
FBX	Filmbox (3D Model File Format)
LOD	Level of Detail
VR	Virtual Reality
AR	Augmented Reality
RPC	Remote Procedure Call
P2P	Peer-to-Peer
GPU	Graphics Processing Unit
FPS	Frames Per Second
Unity	Unity Game Engine
SDK	Software Development Kit

Maintaining Game Fairness Amid Strategic Evolution in Multiplayer Obstacle Course Games

Sunay Kishor Bhojar, Aditya Mittal, Tarun Santani, Pranav Jaju, Prof. Mayur Chavan

¹BE in Computer Engineering, ²BE in Computer Engineering, ³BE in Computer Engineering, ⁴BE in Computer Engineering,

⁵Assistant Professor

¹SCTR's Pune Institute of Computer Technology Pune, India

Abstract - Multiplayer games have gained significant popularity in the contemporary gaming industry due to their unpredictability, competition, and collaboration, setting them apart from traditional storyline games. Titles like Fall Guys and PlayerUnknown's Battlegrounds highlight the growing trend favoring multiplayer experiences. However, the enjoyment of these games can diminish when fairness is compromised, as players naturally develop strategies over time to dominate, which can reduce the fun factor. This evolution of player tactics is inevitable but poses a challenge to maintaining an engaging and balanced experience for all participants. To address this issue, we identified an opportunity to develop a multiplayer obstacle course game while exploring innovative ways to preserve fairness. By introducing dynamic elements that adapt to players' evolving strategies, we aim to create a game that remains exciting, fair, and enjoyable over time, offering a fresh approach to multiplayer gameplay that keeps the competitive edge alive.

Index Terms - Multiplayer games, fairness, unpredictability, collaboration, competition, obstacle course, adaptive gameplay, game strategies, dynamic elements

I. Introduction

A. Background

Multiplayer games have become a pillar of the gaming industry, offering players the chance to compete and collaborate in real-time across various platforms. As the popularity of such games has increased, the need for fairness in gameplay has become increasingly significant. Fairness ensures that players, regardless of their skill level or experience, have an equal opportunity to compete in the games. However, as players evolve their strategies over time, certain dominant play styles emerge that can create imbalances which leads to eradication of fairness in the game.

B. Problem Statement

In strategy-based multiplayer games, players constantly adjust to changing environments and develop better strategies to reach their objectives. Although this evolution is an organic and satisfying part of competitive gaming, it can lead to the emergence of dominant strategies that compromise the fairness of the game. These strategies may provide specific players with advantages, diminishing the enjoyment and competitive drive of others. Balancing the game to ensure a fair and equitable experience for all players while allowing strategic development is a fundamental challenge in the design of multiplayer games.

C. Research Aim

The objective of this research paper is to examine the impact of strategic games, specifically multiplayer obstacle course games, on the development of player strategies and how these strategies can potentially disrupt the fairness of the game. By conducting a thorough analysis of game dynamics and player behavior, we will investigate potential approaches that game developers can take to prevent the emergence of unfair advantages while developing the game from scratch. The focus of the study will be on maintaining fairness without limiting player adaptability by balancing elements such as dynamic difficulty adjustment, matchmaking systems, and randomized obstacles.

II. Literature survey

A. Evolution of Player Strategies and Fairness in Multiplayer Games

In multiplayer games, competition is encouraged, and players naturally develop strategies over time. According to Kristadi et al. [1], a clear UI/UX is important in preventing exploitative gameplay and maintaining fairness. The research by Wynn et al. [2] revealed that multiplayer serious games promote strategic adaptation, but dominant strategies can unbalance the game. Ntelia [3] pointed out that evolving strategies in games like Fortnite can create imbalances that threaten fairness. As players adjust their strategies, game designers must ensure that no single strategy becomes dominant, thus preserving fairness for all participants.

B. Fairness Mechanisms in Game Design

In order to maintain equilibrium, game developers utilize techniques such as dynamic difficulty adjustment (DDA) and randomizing environments. Zhu [4] conducted research on Battle Royale games such as Call of Duty, emphasizing how random barriers and shrinking zones prevent any player from gaining an excessive advantage. This is particularly important in obstacle course games, where unpredictability prevents the memorization of levels. Wang et al. [5] delved into the evolution of cooperation and competition in multiplayer games, demonstrating how fairness can be upheld through effective design. These techniques ensure that dominant strategies do not disrupt gameplay, thus keeping competition fair and captivating.

C. Ensuring Fairness through UX and Matchmaking

The presence of effective user experience (UX) and match-making systems is crucial for ensuring fairness in multiplayer games. Couto and Pal [7] presented decision-making models that enable players to adapt their strategies based on real-time rewards, contributing to equitable gameplay. Matchmaking systems, which match players with similar skill levels, prevent seasoned players from overpowering novices, thereby ensuring fair competition. Engstrom [6] emphasized the importance of collaboration between game designers, software engineers, and UX experts in creating well-balanced and captivating games. By guaranteeing user-friendly interfaces and suitable matchmaking, developers can uphold fairness across diverse player skill levels.

D. Multiplayer Dynamics and Player Evolution

If not properly controlled, player strategy evolution in multiplayer games might compromise fairness. Van Dijk [9] emphasized the need for balanced multiplayer design by studying how multiplayer components in survival games changed the main player experience. Gao et al. [8] explored the principles of environmental and social psychology in UI/UX design for multiplayer games, highlighting the importance of user-centered design in maintaining fairness. Game Analytics [11] demonstrated through successful games like Fall Guys how proper implementation of multiplayer dynamics can create an engaging and fair gaming experience. To solve these challenges, careful design and frequent updates are essential, as noted in various industry analyses [10], [12].

III. Game Development And Fairness Challenges

Ensuring fairness in multiplayer games is crucial to preserving player happiness and participation. Competitive integrity is encouraged in a fair gaming atmosphere where all players, regardless of skill level, believe they have an equal chance of winning. Unfair advantages can cause annoyance and lower player retention, whether they are the result of dominating strategies or imbalanced gameplay features. Fairness is important in game creation for the following main reasons, along with the difficulties that come with it:

- **Player Retention and Enjoyment:** Games that are fair to players of all skill levels continue to be entertaining. Players are more inclined to stick with a balanced game because they believe talent and strategy, not unfair advantages or exploits, are what make them successful. Game that are unbalanced and in which dominating strategies are used might drive away casual or inexperienced players and cause disengagement.
- **Encouraging Competitive Integrity:** Ensuring that competition is founded on skill rather than chance or unbalanced mechanics is crucial for games featuring competitive multiplayer options. In addition to encouraging players to develop their methods without feeling tricked or at a disadvantage, fair competition cultivates a sense of accomplishment. Skilled players frequently take advantage of weaknesses in games with imbalances, which lowers the level of competition and drives away fair players.
- **Preventing Dominant Strategies:** As players develop and adjust to a game, some tactics may grow too strong and upset the equilibrium. Designing game features that exclude any one approach from controlling the games is necessary to ensure fairness. Dynamic difficulty adjustments (DDA) and obstacle layout randomisation, for instance, might prevent dominating strategies from emerging and keep the game interesting and demanding for all players.
- **Preventing Dominant Strategies:** As players develop and adjust to a game, some tactics may grow too strong and upset the equilibrium. Designing game features that exclude any one approach from controlling the games is necessary to ensure fairness. Dynamic difficulty adjustments (DDA) and obstacle layout randomisation, for instance, might prevent dominating strategies from emerging and keep the game interesting and demanding for all players.
- **Player Trust and Game Durability:** Fairness and trust are closely related. Players may stop trusting the game's developers or mechanics if they believe it to be unfair, which could result in unfavourable reviews and a decline in the number of players. Conversely, games that put an emphasis on fairness can foster enduring player loyalty by making players feel appreciated and secure in the competition's integrity. Throughout the game's existence, a robust and engaged player base depends on this trust.
- **Anti-Exploitation Measures:** Fairness can be seriously compromised by cheating or abusing game mechanics. Anti-cheat software is essential for stopping gamers from using in-game exploits or outside tools to obtain unfair advantages. These safeguards guarantee that every participant is competing on an equitable playing field in addition to preserving the integrity of the game.

IV. Solution For Ensuring Fairness

To ensure fairness in multiplayer games, a variety of tactics must be implemented, including those that balance gameplay, prevent dominating strategies, and promote equitable competition. The following are some suggestions for preserving equity:

A. Dynamic Difficulty Adjustment (DDA)

By changing the game's complexity according to a player's performance, Dynamic complexity Adjustment (DDA) contributes to the preservation of fairness. In order to challenge a player who performs extraordinarily well, the difficulty is raised; for lesser players, the difficulty is decreased. This promotes a more competitive atmosphere by ensuring that no participant has a significant edge.

B. Randomized Level Design

Each round's level design, obstacles, and game elements are randomly chosen to keep players from learning layouts and becoming dependent on the same tactics. As a result, players are forced to adjust and no single tactic can dominate every round, keeping the game surprising and interesting.

C. Balanced Matchmaking Systems

By matching players with comparable skill levels, fair matchmaking keeps more experienced players from controlling less experienced ones. Matchmaking systems can guarantee that players encounter opponents who are equally difficult by classifying players according to their performance indicators, which will make games more equitable and pleasurable for all.

D. Anti-Cheat Systems

Cheating undermines fairness by providing some participants an unfair edge over others. Anti-cheating methods, such as real-time monitoring and detecting unauthorised software, contribute to a level playing field. These features are critical for maintaining the game's integrity and preventing mechanics from being exploited.

E. Playtesting and Feedback Loops

Regular playtesting provides useful insights on game balance. By soliciting feedback from users, developers might detect possible fairness flaws, such as overpowering strategies or systems. This feedback helps to improve the game and ensure that it remains fair as it evolves.

F. Dynamic Power-Ups and Scoring Systems

Incorporating dynamic power-ups and performance-based rewards can help to balance gameplay. Players who are struggling can receive tiny boosts, while those in the lead may face greater hurdles. A well-designed scoring system that rewards competence and adaptability promotes a balanced and fair experience.

V. Implementation in Our Multiplayer Obstacle Course Game With Our Proposed Solution for Fairness

In addition to typical fairness methods, such as balanced matchmaking and randomised obstacle layouts, an innovative approach to increasing fairness in the online obstacle course game might be the introduction of Adaptive Game Narratives. This system dynamically alters the game's narrative based on player performance and behavior, ensuring that no single player or strategy becomes dominant while maintaining an immersive experience.

A. Adaptive Game Narratives

The proposal proposes a system in which each player's in-game experience is slightly adjusted to their current standing or playstyle. For example, players that consistently finish at the top may encounter more surprising difficulties or challenges in subsequent rounds, whilst struggling players receive subtle aid, such as clearer clues about approaching obstacles or little movement speed enhancements in specific portions.

1) Key Features::

- **Performance-based Obstacle Difficulty:** As players progress, the game dynamically adjusts obstacle complexity based on real-time performance. Top-performing players may face more precise hurdles, whereas others experience more forgiving challenges, ensuring a level playing field.
- **Game Environment Adjustment:** The in-game environment (e.g., wind, gravity, terrain friction) could be adjusted subtly based on a player's overall performance, preventing a snowball effect in which one player dominates the game.

- **Strategic Power Rebalancing:** Instead of standard power-ups, select players may be granted "strategic choices" during critical moments in the game. For example, those trailing behind may be granted shortcuts or alternate pathways, but leading players may confront more branching paths to preserve balance.
- **Narrative Rewards for Creativity:** To encourage strategic diversity, players who choose less popular routes or employ imaginative techniques may be rewarded with narrative aspects (such as secret passages or bonus content) that improve their gaming without directly increasing their chances of winning.

B. Benefits for Fairness:

- **Keeps Players Engaged:** By introducing narrative-based adaptability, even less-skilled players remain engaged and feel they have a chance to succeed.
- **Prevents Dominant Strategies:** Since the narrative dynamically adjusts, it discourages players from relying on a single dominant strategy throughout the game.
- **Balancing through Creativity:** By rewarding diverse playstyles, the game incentivizes creativity rather than simply rewarding speed or precision, creating a more balanced and enjoyable experience for all players.

VI. CONCLUSION

Ensuring fairness in multiplayer games is an important part of game design since it affects player involvement, enjoyment, and retention. In this paper, we looked at how player strategies change over time, potentially leading to dominating play styles that upset game balance. To solve these difficulties, several fairness solutions were proposed, including Dynamic Difficulty Adjustment (DDA), randomised level designs, balanced matching, and anti-cheat systems.

In addition to these traditional methods, we introduced the concept of *Adaptive Game Narratives*, a novel approach where the game dynamically adjusts its narrative and difficulty based on player performance. This approach promotes fairness by providing adaptive challenges and opportunities for all players, regardless of skill level.

While this strategy introduces novel ways to improve fairness, it also has certain drawbacks, such as potential unhappiness among competent players, balancing challenges, and implementation complexity. However, with careful design and constant playtesting, these issues can be addressed, resulting in a more balanced and engaging experience for all players.

Overall, by using these tactics and addressing limits, game developers may create a fair and competitive environment for online games, fostering long-term user pleasure and engagement.

VII. ACKNOWLEDGMENTS

We are grateful to the Pune Institute of Computer Technology (PICT) for providing us with the chance and resources to work on this research challenge. Their ongoing support has been vital in guiding the course of our research and development.

We are also appreciative to the writers and researchers whose work we have cited in this publication. Kristiadi et al. (2017), Wynn et al. (2017), Ntelia (2017), Zhu (2023), Wang et al. (2024), and others all made substantial contributions to our understanding of fairness mechanisms in multiplayer games. Their research has served as a solid foundation for developing innovative ways to ensure balanced gameplay.

VIII. REFERENCES

- [1] Kristiadi, D.P., Udjaja, Y., Supangat, B., & Prameswara, R.Y. (2017). "The effect of UI, UX and GX on video games". In Proceedings of the 2017 International Conference on Cybernetics and Computational Intelligence (CYBERNETICSCOM) (pp. 41-45). IEEE. <https://doi.org/10.1109/CYBERNETICSCOM.2017.8311702>
- [2] Wynn, A., Wang, J., & Hsu, T.C. (2017). Multiplayer Serious Games Supporting Programming Learning. In Proceedings of the European Conference on Games Based Learning (pp. 41-45). <https://doi.org/10.34190/ecgbl.17.1.1621>
- [3] Renata Ntelia. Fortnite as Bildungsspiel? Battle Royale Games and Sacrificial Rites. In Researchgate articles (pp. 41-45).
- [4] Zhu, T. (2023). Design implementation and performance optimization of battle royale games. In Proceedings of the International Conference on Global Politics and Socio-Humanities. <https://doi.org/10.54254/2753-7064/17/20230833>
- [5] Wang, C., Perc, M., & Szolnoki, A. (2024). Evolutionary dynamics of any multiplayer game on regular graphs. Nature Communications, 15(1), Article 49505. <https://doi.org/10.1038/s41467-024-49505-5>
- [6] Engström, H. (2020). Game development research. Retrieved from <https://www.researchgate.net/publication/348446595>

- [7] Couto, M. C., & Pal, S. (2023). Introspection dynamics in asymmetric multiplayer games. *Dynamic Games and Applications*, 13(4), 1256–1285. <https://doi.org/10.1007/s13235-023-00525-8>
- [8] Gao,B.,Xie,H.,Wang,Y.,&Xie,Z.(2024).Principlesofenvironmen- tal and social psychology in UI/UX design for metaverse social games: A case study of Horizon Worlds. *Environment and Social Psychology*, 9(1). <https://doi.org/10.54517/esp.v9i1.1871>
- [9] van Dijk, F. (2024). Implementing multiplayer in single-player games: Analysis of multiplayer integration in single-player survival games [Thesis]. Breda University of Applied Sciences. <https://doi.org/10.13140/RG.2.2.33797.36323>
- [10] VG Insights. (n.d.). What makes *Fall Guys* so special. Retrieved from <https://vginsights.com/insights/article/what-made-the-fall-guys-so-successful>
- [11] Game Analytics. (n.d.). How *Fall Guys* did it differently. Retrieved from <https://gameanalytics.com/blog/everything-you-can-learn-from-fall-guys-ultimate-knockdown/>
- [12] Pe´rez, S. (2020, September 4). From the developers of *Fall Guys*. *The Washington Post*. Retrieved from <https://www.washingtonpost.com/video-games/2020/09/04/fall-guys-lore-crunch-levels/>

